RETROENGINE BETA ENCYCLOPEDIA

Volume 5: Dedicated Modding & Extensibility Technical Guide

1. DEDICATED MODDING ARCHITECTURAL PHILOSOPHY

RetroEngine's dedicated modding support is modeled directly after the deep, programmatic extensibility of **Minecraft (Forge/Fabric)** and Bethesda's **Creation Engine (Skyrim/Fallout)**. Rather than offering shallow XML configuration tweaks or restricted Lua sandboxes, RetroEngine leverages Java 25 Virtual Threads, Janino in-memory bytecode compilation, dynamic ECS schema registration, and 128-bit UUID Virtual File System (VFS) stacking. Modders can inject real gameplay logic, define custom components, override asset pipelines, apply non-destructive level delta patches, and author GPU shaders.

2. VIRTUAL FILE SYSTEM (VFS) OVERLAY STACKING

To support seamless resource overriding without corrupting core engine assets, AssetDatabase implements a priority-stacked Virtual File System. When the engine requests an asset by project path or 128-bit UUID, the VFS searches top-down across active mod directories before falling back to the base Assets/ folder.

VFS Layer	Directory Path	Priority Level	Override & Resolution Behavior
`Active Mod N`	`Mods/02_SuperWeapons/Assets/`	**High (Priority 200)**	First lookup target. Overrides lower mods and vanilla assets.
`Active Mod 1`	`Mods/01_HDTextures/Assets/`	**Medium (Priority 100)**	Evaluated if Mod N does not supply the requested path/UUID.
`Vanilla Core`	`Assets/`	**Base (Priority 0)**	Fallback root. Provides core textures, meshes, and scenes.

CRITICAL ARCHITECTURAL GUARANTEE

128-Bit UUID Hijacking: When a mod places a file at Mods/MyMod/Assets/Textures/Wall.png, the VFS binds it to the exact 128-bit UUID assigned to vanilla Textures/Wall.png. Base level scenes (.rscene) and prefabs (.rprefab) automatically display modded assets without requiring manual scene file modifications.

3. MOD DISCOVERY & MANIFEST SPECIFICATION (mod.json)

On engine initialization, ModLoader scans the Mods/ directory for mod sub-folders containing a valid mod.json manifest file. Manifests define load priorities, dependency trees, target engine versions, and custom entry point scripts.

```
{
  "mod_id": "super_weapons_pack",
  "name": "Super Weapons & Plasma Cannon Mod",
  "version": "1.4.0",
  "author": "RetroModder",
  "target_engine": "0.9.5-beta",
  "load_priority": 150,
  "dependencies": ["core_lib_mod"],
  "entry_point": "com.retro.mod.SuperWeaponsMod",
  "script_sources": ["Scripts/PlasmaCannon.java", "Scripts/WeaponMod.java"],
  "asset_overlays": ["Assets/"]
}
```

4. TOPOLOGICAL LOAD ORDER RESOLUTION

ModLoader processes all discovered mod manifests using a topological sorting algorithm based on explicit dependencies and numerical load priorities. If a dependency is missing or cyclic, ModLoader flags a clear diagnostic in EngineConsolePanel and safely unloads the dependent mod.

ModLoader Pipeline Step	System Executed	Functional Task
`1. Discovery Scan`	`ModLoader.discoverMods()`	Iterates Mods/ folder, parses mod.json manifests, builds dependency graph.
`2. Topological Sort`	`ModLoader.resolveLoadOrder()`	Orders mods to satisfy dependencies and priority rankings.
`3. VFS Mount`	`AssetDatabase.mountModVFS()`	Pushes mod Assets/ directories onto the VFS stacked lookup priority queue.
`4. Janino Script Build`	`ScriptEngine.compileModScripts()`	Compiles mod .java files in memory and instantiates entry points.

5. DYNAMIC IN-MEMORY JAVA SCRIPT COMPILATION

Modders write standard Java code extending ModScript or EntityScript. RetroEngine uses Janino's SimpleCompiler to compile source code directly into JVM bytecode at runtime. Modders do not need external compilers, Gradle setups, or C++ toolchains.

```
package com.retro.mod;

import com.retro.engine.scripting.EntityScript;
import com.retro.engine.scene.entity.Entity;

public class PlasmaCannonScript extends EntityScript {
    public float ammoCapacity = 100.0f;
    public float fireRate = 0.1f;

    @Override
    public void onAwake() {
        logInfo("Plasma Cannon initialized on entity: " + (entity != null ? entity.name : "Unknown"));
    }

    @Override
    public void onUpdate(float deltaTime) {
        if (getTransform().posY < -10.0f) {
            setPosition(0.0f, 5.0f, 0.0f); // Auto-respawn safety
        }
    }
}
```

6. RUNTIME EXCEPTION ISOLATION & HOT-SWAPPING

If a modder's script throws a runtime exception (e.g., NullPointerException or divide-by-zero) during onUpdate() or onPhysicsUpdate(), RetroEngine's exception guard catches the error, logs a full stack trace to EngineConsolePanel, disables that script instance, and keeps the main engine loop ticking cleanly. Modders can fix the script file on disk; the editor watcher detects the save and hot-reloads the script instantly.

7. HIGH-PERFORMANCE IN-MEMORY EVENT BUS (ModEventBus)

Deep modding requires modders to hook into global engine events—such as entity spawning, collision manifold creation, scene level loads, and damage processing. RetroEngine provides ModEventBus, a zero-allocation event dispatch framework.

Engine Event Class	Trigger Conditions	Mod Modification Capabilities
`EntitySpawnEvent`	Fired when an entity is instantiated via scene load or prefab spawn.	Cancel spawn, inject modded components, or alter transform scales.
`CollisionEvent`	Fired on physics manifold generation in NarrowphaseSolver.	Modify contact impulses, apply elemental damage, or trigger particle audio.
`TriggerEvent`	Fired when an entity enters/exits a yellow trigger volume.	Execute custom quest triggers, warp scenes, or give modded items.
`SceneLoadEvent`	Fired immediately after scene binary deserialization.	Inject custom lighting, apply delta patches, or spawn persistent mod entities.
`RenderPassEvent`	Fired during SceneRenderPipeline pass execution.	Issue custom draw commands or bind custom mod post-processing shaders.

```
public class QuestModListener {
    @SubscribeEvent
    public void onEntitySpawn(EntitySpawnEvent event) {
        if ("BossMonster".equals(event.entity.name)) {
            // Attach modded BossHealthComponent dynamically
            event.entity.addComponent(new BossHealthComponent(5000.0f));
        }
    }
}
```

8. DYNAMIC ECS COMPONENT SCHEMA REGISTRATION

Surface-level engine modding restricts users to editing vanilla parameters. RetroEngine allows modders to define and register **entirely new ECS Component types** at runtime. Registered mod components automatically gain binary serialization and visual editor UI support.

```
// 1. Modder defines a custom Component class with @InspectorProperty annotations
public class ShieldGeneratorComponent extends Component {
    @InspectorProperty(label = "Shield Capacity", min = 0.0f, max = 1000.0f)
    public float maxShield = 250.0f;

    @InspectorProperty(label = "Recharge Rate (Units/Sec)")
    public float rechargeRate = 15.0f;

    @InspectorProperty(label = "Is Shield Active")
    public boolean isActive = true;
}

// 2. ModLoader registers component schema during boot
ComponentSchema.registerModComponent(ShieldGeneratorComponent.class);
```

9. AUTOMATIC BINARY SERIALIZATION & INSPECTOR REFLECTION

Because ComponentBinaryCodec reads component field schemas dynamically and InspectorPanel reflects public fields, modded components serialize into .rscene files seamlessly and generate native numeric steppers, switches, and asset selectors in the Visual Editor.

10. NON-DESTRUCTIVE SCENE DELTA PATCHING (.rpatch)

To prevent multiple mods from corrupting the core Main.rscene level file when adding content to the same world, RetroEngine introduces a Creation Engine-style delta patching format: .rpatch. Mods store relative delta instructions instead of full scene files.

Patch Delta Command	Target Entity Resolver	Runtime Modification Executed
`ADD_ENTITY`	`New Entity ID / UUID`	Instantiates a new modded entity or prefab into the active SceneWorld.
`MODIFY_COMPONENT`	`Entity 128-Bit UUID`	Mutates specific fields on an existing component (e.g. changing Light color).
`REMOVE_ENTITY`	`Vanilla Entity UUID`	Marks a vanilla entity as disabled/destroyed without modifying vanilla files.
`ATTACH_COMPONENT`	`Target Entity UUID`	Dynamically attaches a modded component to a vanilla entity graph.

CREATION ENGINE SCENE DELTA PRINCIPLE

Multi-Mod Compatibility: If Mod A changes a door's lock script and Mod B adds a treasure chest nearby, both mods ship lightweight .rpatch files. On level load, SceneSerializer loads vanilla Main.rscene, then applies Mod A's delta, followed by Mod B's delta in load order. Zero file collisions occur!

11. CUSTOM MOD SHADERS & GRAPHICS INJECTION

Modders can provide custom GLSL shaders (.vert, .frag) for Vulkan 1.3 or Metal shading language files (.metal) for macOS. Shaders placed in a mod's Shaders/ directory override standard materials or register custom post-processing passes.

Shader Pipeline Target	Supported Formats	RHI Re-Compilation Mechanics
`Vulkan 1.3 Backend`	`GLSL 4.60 (.vert / .frag)`	Compiled to SPIR-V bytecode in background thread via glslc.
`Metal 3 Backend`	`Metal Shading Language (.metal)`	Compiled to AIR bytecode via native macOS xcrun metal compiler.
`Fallback Registry`	`ShaderSources.java`	Provides embedded core string shaders if mod shader build fails.

12. ZERO-DOWNTIME SHADER RE-COMPILATION

When a modder updates a shader file, VulkanShaderCompiler attempts a background build. If compilation succeeds, IRenderBackend.reloadShaders() hot-swaps the active pipeline state object (PSO) on the next frame boundary. If a syntax error occurs, the error is logged to EngineConsolePanel, and the RHI safely retains the previous valid pipeline.

13. MOD ASSET COOKING & .rpack VRAM PACKAGING

While raw .png textures and .obj meshes can be loaded directly during mod development, release mods can be cooked using RpackCookerCli into binary .rpack VRAM packages. This pre-swizzles textures into GPU-native compressed formats (BC3 / ASTC) and pre-indexes mesh vertex buffers.

Asset Type	Raw Input Format	Cooked .rpack Binary Format
`2D Textures`	`PNG, JPG, TGA (Uncompressed)`	BC3 (Desktop Vulkan) / ASTC 8x8 (Apple Metal UMA) Texels
`3D Meshes`	`OBJ, FBX, GLTF (Text / JSON)`	Pre-Indexed Interleaved Vertex Arrays & Bounding Box AABBs
`Skeletal Animations`	`FBX / Collada Joints`	Quantized 64-Matrix Bone Keyframe Tracks (.ranim)

14. JDK 25 VIRTUAL THREAD ASSET STREAMING

Modded assets are streamed asynchronously via AssetStreamer using JDK 25 Virtual Threads. High-resolution mod textures and large models load in the background without causing frame stutters or locking the main render loop.

15. CREATION KIT MODE: EDITOR SUITE AS A MODDING TOOL

Following the philosophy of Bethesda's Creation Kit, RetroEngine's Visual Editor (EditorWorkspace, DockSpace, GizmoSystem) is shipped alongside the game as an official modding tool. Modders create content in the exact same workspace used to build the vanilla game.

Editor Feature	Vanilla Developer Use	Modder Creation Kit Function
`Project Launcher`	`Loads vanilla Game_Data/`	Selects active mod workspace (Mods/MyMod/) with vanilla data read-only.
`3D Viewport & Gizmos`	`Builds base levels & prefabs`	Places modded entities, edits colliders, and saves .rpatch scene deltas.
`Animator Graph`	`Authors character state machines`	Adds custom animation states, blend trees, and parameter transitions.
`Build Exporter`	`Exports standalone executables`	One-click packages mod assets into .rpack archives and outputs .rmod.

CREATION KIT DISTRIBUTION OVERLAY

One-Click Steam Workshop Exporter: In BuildExporterOverlay, modders click **'Package Mod for Distribution'**. The editor validates script syntax, cooks assets into .rpack archives, generates the manifest, and packages a clean .rmod distribution file ready for uploading to Steam Workshop or NexusMods.

16. COMPLETE MODDING API SIGNATURE MATRIX

Summary reference table of core Java classes and API signatures available to mod developers:

Class / Subsystem	Key API Methods	Mod Functional Scope
`EntityScript`	`getTransform(), getComponent(), raycast(), loadScene()`	Base class for attached entity logic and physics hooks.
`ModScript`	`onModInit(), onModEnable(), onModDisable()`	Base entry point for global mod lifecycle execution.
`ModEventBus`	`register(), unregister(), post(Object event)`	Zero-allocation event bus for subscribing to engine events.
`ComponentSchema`	`registerModComponent(Class clazz)`	Registers custom ECS component schemas for UI & serialization.
`AssetDatabase`	`mountModVFS(String path), resolveAsset(String path)`	Pushes mod directories onto VFS priority lookup stack.
`SceneWorld`	`spawnEntity(), findEntityByName(), raycastExcluding()`	Queries active scene graph and performs spatial raycasts.

17. BETA MODDING INTEGRATION & STABILITY CHECKLIST

1. **VFS Isolation:** Verify mod asset overrides do not corrupt base Assets/ files.
2. **Exception Safety:** Ensure mod script errors log to EngineConsolePanel without stopping EngineLoop.
3. **Multi-Mod Delta Testing:** Validate multiple .rpatch files load cleanly in sequence.
4. **VRAM Memory Sweeping:** Verify unreferenced mod assets are evicted after 600 frames.